

Instituto Tecnológico de Costa Rica.

Escuela de Ingeniería en Computación.

Taller de programación

Documentación Sudoku ver:2.0 2026

Profesor: William Mata Rodríguez.

Autor:

Ezequiel Bonilla Vega 2025098474

Escuela de Ingeniería en Computadores.

Fecha:

Versión 1: 07/06/2026

Versión 2: 28/06/2026

I semestre

Contents

Contenido: 3

 Definición del proyecto: 3

 Enunciado del proyecto:..... 3

 Objetivos del proyecto:..... 3

 Implementación:..... 4

Marco teórico:..... 11

 Temas investigados:..... 11

 Estructura archivos usados: 15

Conclusiones del trabajo: 15

 Problemas encontrados y soluciones implementadas..... 15

 Estadística tiempos:..... 17

 Lista revisión proyecto:..... 18

Anexos: 20

Referencias: 27

Contenido:

Definición del proyecto:

“Sudoku 2026” es el segundo proyecto del curso de Taller de programación, el cuál consiste en un programa capaz de ejecutar partidas de Sudoku en diferentes dificultades con el objetivo que el usuario complete los tableros. Además posee diferentes configuraciones que permiten adaptar las partidas al estilo de juego de cada jugador, como las diferentes opciones de tiempo o la opción de guardar y cargar partidas.

La versión 2.0 del programa corresponde a una revisión del programa, aplicación de mejoras e implementación de características nuevas como un sistema de usuarios y verificación de la cuenta con encriptación de un código.

Enunciado del proyecto:

Crear un programa que contenga una IGU (Interfaz gráfica de usuario) que sea capaz de ejecutar diferentes partidas de Sudoku. Se usarán botones para acceder las funcionalidades del programa, es decir, lo que el programa hace.

Agregar la funcionalidad de cuentas para acceder a la aplicación, con verificación de identidad por correo y mayor variedad de configuración al usuario.

Objetivos del proyecto:

- Aplicar y reforzar aspectos del lenguaje Python 3 como el uso de diversos componentes, manejo de técnicas de iteración además del manejo de diversas estructuras de datos.
- Uso de programación estructurada.
- Aplicar buenas prácticas de programación: documentación interna y externa del programa, nombres significativos, desarrollo y reutilización de funciones, seguir un estilo de programación.
- Validar los datos de entrada.
- Usar archivos de datos qué son y su importancia.
- Fomentar algunas habilidades investigativas

- Programación dirigida por eventos (event-driven programming) en el desarrollo de software.
- Interfaz gráfica de usuario (GUI: Graphical User Interface) en el desarrollo de software y su desarrollo en Python con Tkinter.
- Realizar la actividad de mantenimiento de software
- Implementar el tipo de datos abstracto Árbol Binario de Búsqueda
- Aplicar conceptos de programación orientada a objetos (POO): clases, atributos, métodos y objetos
- Usar software de control de versiones y trabajo colaborativo: Github.

Implementación:

“Sudoku 2026” es, como se mencionó anteriormente una implementación de software en donde su principalidad funcionalidad es ejecutar partidas de sudoku y contener diferentes opciones de accesibilidad de cara al usuario. A continuación se explorará la implementación que fue llevada a cabo para lograr representar los diferentes objetivos y como se lograron plasmar dentro del programa como tal.

Para este programa se utilizó Python 3 como lenguaje de programación en el IDE Visual Studio Code, todo el programa se encuentra dentro de la carpeta Programa_II y contiene los archivos para su ejecución:

- main.py, game.py, config.py y JSONParser.py que corresponden a los archivos escritos en el lenguaje Python que ejecutan como tal el juego.
- sudoku2026_bitácora_jugadas.json, sudoku2026juegoactual.json y sudoku2026partidas.json que corresponden a los archivos de configuración y almacén de datos del programa, todos en formato JSON.
- Finalmente, diversos archivos PDF, entre ellos este documento, el manual de usuario del programa y el output de los mejores jugadores (Él cuál es una funcionalidad del programa)

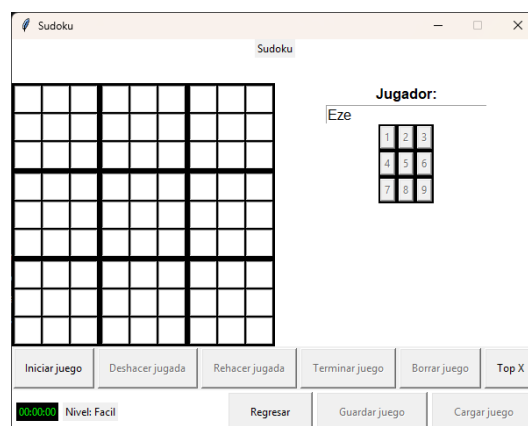
Para lograr ejecutar el juego iniciando por una pantalla de configuración, dentro de la carpeta “vscode” se creó un archivo llamado launch.json que contiene las instrucciones de ejecución del programa, en este se configuraron

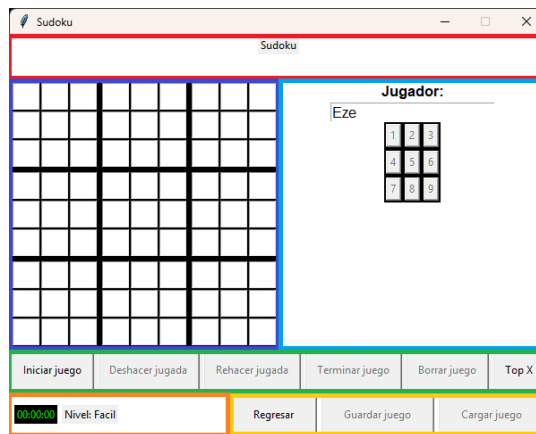
diferentes opciones de configuración como la posibilidad de ejecutar archivos individuales destinado a pruebas durante la creación del código, así como ejecutar “main.py” como inicio del programa que corresponde a la ejecución estándar de todos los usuarios.

El programa cuenta con una Interfaz gráfica de Usuario o IGU por su siglas (El cuál se le seguirá refiriendo de esta manera), la diferencia con interfaces de otro tipo como CLI (Command Line) es la capacidad del usuario de utilizar el mouse para acceder a las funcionalidades del programa por medio de “Widgets” que sirven leen el input del click y ejecutan su lógica interna para realizar las acciones necesarias, sin la necesidad de comprobar que la opción del usuario esté bien digitada.

La biblioteca utilizada para este fin es Tkinter, esta permite crear una gran gama de interfaces gráficas según lo que necesite el usuario. Un programa de Tkinter contiene una ventana donde se pueden colocar los diferentes componentes como botones, entradas de texto, etiquetas, entre otros. Así como paneles y diferentes opciones de configuración que permiten agrupar y ordenar componentes.

Cada una de las interfaces fue pensada como una serie de bloques, cada bloque contiene una serie de componentes dentro, esto permite, como se mencionó anteriormente, lograr acomodarlos a gusto del desarrollador, a continuación, un ejemplo:





Dentro de los componentes utilizados tenemos botones, que detectan el input del mouse y ejecutan acciones en consecuencia, label los cuales permiten colocar texto además de darles formato adicional, entry el cuál corresponde a una entrada de texto; el usuario puede utilizar el teclado para insertar texto. Finalmente Frame que son los paneles anteriormente mencionados.

Las tres ventanas visibles corresponden a sus diferentes archivos .py. Cada una utiliza Tkinter para la interfaz únicamente. Además, de utilizar JSONParser para manipulación de los diferentes archivos del tipo JSON. Para la implementación de lectura y escritura de estos archivos se hizo uso de la librería json nativa de Python.

JSONParser es dinámico, por lo cuál funciona para cualquiera de los archivos JSON dentro del programa, el código sigue una serie de reglas que permite esta versatilidad, primero las funciones de lectura y escritura están separadas de la lógica de los datos, esto permite que diferentes archivos con diferentes datos puedan ser procesados por las mismas funciones.

A partir de ahí según el archivo con el que se esté trabajando, por ejemplo, bitácora o partidas guardadas, determina que funciones ejecutar, registrar partida y guardar y cargar partidas respectivamente. La ventaja de estas funciones y la integración de botones, permite que cada botón ejecute solo lo que le corresponde sin tener que validar la entrada:

- Botón guardar: Llama a la función guardar_partida de JSONParser y finalmente la genérica de read y sabe.

- Botón cargar: Llama a la función cargar_partida de JSONParser y finalmente llama a la función genérica de read.
- Al completarse la partida: Llama a la función registrar_partida de JSONParser y finalmente la genérica de read y sabe.

Para implementar el tablero de Sudoku se implementó una matriz 9x9, para realizar esto en Python se utilizaron listas, una lista principal que actúa como contador de filas y listas internas que funcionan para el contador de columnas. Los datos de esta son exclusivamente números del tipo enteros, representando el valor del 1 al 9 y -1 para casillas vacías.

El programa interpreta la opción de “números” o “letras” pero nunca transforma la matriz de números a letras. El programa lee si la opción de letras está habilitada y según el valor de la casilla lo consulta en un diccionario donde las llaves son los números del -1 al 9 (Exceptuando el 0) y los valores las letras como tal.

No solo se utilizaron los diccionarios y listas nativas de Python para estos apartados; Los diccionarios también se hicieron uso en el manejo de datos, en donde ya sea para guardar o cargar datos se pasaba un diccionario con las respectivas clave valor para acceder a estos, esta implementación facilitó la manera en la que el programa leía los datos, evitando utilizar ciclos for o while que podrían hacer más lenta la ejecución.

Las listas, por su parte, se utilizaron para el registro de jugadas, donde actuaban como una pila, a diferencia de utilizar una librería especial, era más viable esta implementación ya que partiendo de la lógica de las listas, solo necesitamos acceder al primer elemento en todo momento, lo cual podemos hacer fácilmente con índices (Ejemplo: pila[0]) tomando en cuenta que hay que eliminar el elemento una vez lo obtenemos (Operación pop). Al ser una estructura ampliamente usada durante varios aspectos del programa, sino solo en dos situaciones particulares se decidió tomar este enfoque.

Para el PDF se hizo uso de la librería fpdf instalada, la cual nos permite trabajar con archivos pdf, tanto transformar datos y crear archivos (Como fue

usado en esta implementación) así como la lectura de datos de este. Se implementó el uso de esta librería en la función `crear_pdf()`, su funcionamiento es la de crear columnas de texto, estas se distribuyen en filas según el tamaño que se defina, existen ciclos para leer los datos del archivo “sudoku2026_bitácora_jugadas” el cuál contiene todas las partidas ganadas por usuario en diferentes dificultades.

Para la lectura del mismo se utiliza las funciones anteriores de `read()` de la clase `JSONParser`, además de las funciones nativas de Python para ordenar los datos del diccionario. Luego se pasan los datos que se dibujan por línea y se ejecutan y dibujan una cantidad de datos `N`, donde `n` corresponde a la opción configurada por el usuario para generar entradas en el archivo de top. Por otra parte, para acceder a los directorios del sistema se utilizó `os`, dentro del mismo entorno de Python. Esto nos permite abrir el archivo una vez creado.

Para la actualización del programa en la versión 2.0 se implementaron diferentes clases extra que ayudaron a realizar las funcionalidades extra del sistema, entre ellas están:

- `GeneradorTablero.py`: Crea por medio de un algoritmo diferentes tableros de sudoku en donde dependiendo de la dificultad, oculta x cantidad de elementos.
- `Mail.py`: Encargada de enviar los correos a las diferentes direcciones con el código.
- `PickleParser.py`: Encargada de leer, modificar y guardar el archivo `.pkl` que contiene los mejores tiempos de las partidas.
- `User.py`: Una clase que contiene la información del usuario, permite crear y consultar usuarios rápidamente.
- `ABB.py`: El árbol binario BST (Por sus siglas en inglés) que utiliza la aplicación para almacenar los tiempos de los juegos completados.

A continuación una descripción de cada uno de ellos además de ampliar en sus funciones dentro del programa.

El marco de las interfaces se mantiene igual, desplegándose sudoku.py al iniciar, solamente que ocultándose inmediatamente para llamar la ventana del login, esta ventana cuenta con dos entradas de texto para ingresar correo y código de verificación. Inicialmente se le solicita al usuario que ingrese el correo para posteriormente activar el campo para ingresar el correo.

Durante esta activación, la aplicación hace uso de la clase mail.py esta se inicializa con el correo del campo de texto así como de un código generado de 6 dígitos, luego llama a la función enviar_correo() dentro de su constructor, luego de establecer el puerto y el tipo de protocolo.

Esta función se encarga de redactar un mensaje con asunto y como contenido un texto además del código de 6 dígitos para que el usuario lo ingrese dentro de la aplicación. Una vez el botón de verificar código es pulsado, este convierte la entrada del campo de texto en un hash codificado y pasado a hexadecimal, este se compara con el código almacenado dentro de los datos del usuario ya codificado, si los dos coinciden se le otorga acceso al resto de la aplicación, apareciendo la pantalla principal.

Otros cambios del programa se realizaron con respecto a la configuración, agregando dos entradas adicionales que serían el juego multinivel y los elementos definidos por el usuario “*costum*”. Estos elementos *costum* se almacenan dentro de los datos del usuario y si este ya tenía previamente creados alguno estos se cargan directamente.

Al seleccionar la opción de costum y al aceptar crear nuevos elementos, el usuario podrá guardar qué caracteres se utilizarán dentro del juego, estos se guardan en un diccionario con las llaves siendo los números del 1 al 9 y el -1 que indica casilla vacía, esto para facilitar la lectura a la hora de colocar dichos elementos.

Una vez se guarden o no los cambios se pasa como parámetro al juego, un diccionario con los diferentes parámetros como el tipo de reloj, la modalidad/ nivel, así como el tiempo y otros detalles adicionales; además de la clase usuario que se crea a partir de lo cargado en el archivo json al iniciar sesión.

Otra de las características incluidas fueron la generación aleatoria de tableros y la implementación de un botón de claves o pistas, ambas estrechamente relacionadas. La clase encargada de generar los tableros es `GenerarTablero.py`, esta crea un objeto donde contiene un tablero lleno que utiliza la recursividad para llenar la matriz a partir de una lista de elementos aleatorios generados del 1 al 9. Este se pasa por una función que genera el tablero final con espacios vacíos sin modificar la matriz original. Esto permite consultarla en cualquier posición para saber el valor que le corresponde, lo cuál hace el botón de clave.

Finalmente, la función del botón Top X, hace lo mismo que la versión pasada, pero leyendo el archivo pickle. Dentro de este solo está la clase ABB, un árbol binario que contiene todas las partidas, esto es ventajoso por dos motivos: primero, el archivo codifica los datos por lo cuál si se abre el archivo no se puede determinar que significa cada apartado a simple vista, a diferencia de json. Por otra parte no es necesario pasar los datos únicamente, ya que este módulo de Python puede codificar clases.

Esto es posible gracias a que Pickle codifica las instrucciones para crear el dato, no el dato en si. Lo que permite guardar y cargar clases fácilmente sin pasar por ciclos que interpreten y reescriban los datos. De esta manera `PickleParser` logra toda esta hazaña.

Marco teórico:

Temas investigados:

- **Interfaz Gráfica de Usuario (GUI, Graphic User, Interface):** Interfaz Gráfica de Usuario: Son los elementos gráficos que nos ayudan a comunicarnos con un sistema o estructura. ...tiene características de hipertexto, lo que permite un entendimiento y/o acción por parte del usuario. (Luna González, 2004)
- **Programación dirigida por eventos (Event-based programs) :** ...Se encuentran organizados alrededor del procesamiento de eventos. Cuando un programa no puede completar inmediatamente porque espera que ocurra un evento... este registra una “callback” (Una función que se invoca cuando suceda el evento). Dabek (2002)
- **Archivos de datos:** Un archivo de datos contiene datos reales o una vista de los datos. Los registros de los archivos de datos se agrupan en miembros. Todos los registros de un archivo pueden estar en un miembro o se pueden agrupar en miembros diferentes. (IBM, 2024).
- **Archivos JSON:** formato basado en un subconjunto de la sintaxis de JavaScript: literales de matrices y objetos, o sea, soporta dos tipos de estructuras, una de ellas los objetos que contienen una colección de pares llave-valor y el otro, arreglos de valores. (Pompa Sourd, (2022), p.5).
- **Tkinter:** Interfaz de Python para la biblioteca GUI de Tk y ha formado parte de la biblioteca estándar de Python desde 1994 con el lanzamiento de la versión 1.1 de Python, convirtiéndose en la biblioteca GUI de facto para Python. (Moore, 2018, p.).
- **PDF:** Un archivo en formato .pdf (del inglés *portable document format*, formato de documento portátil) es un tipo de archivo creado a partir de otro archivo... es un formato de archivo universal que conserva las fuentes, las imágenes y la maquetación de los documentos originales

creados en una amplia gama de aplicaciones y plataformas. (Estandarte, 2026, párr. 3).

- **ABB o BST:** Un árbol binario es un tipo de estructura de datos en forma de árbol donde cada nodo puede tener un máximo de dos nodos hijos: un nodo hijo izquierdo y un nodo hijo derecho...resultan más fáciles de entender, de implementar y se ejecutan con mayor rapidez. (W3Schools, s.f.).
- **Módulo Pickle:** ¿Qué almacena el modulo *Pickle*? Todos los tipos de datos nativos que soporta Python..., sets que contienen alguna combinación de datos nativos... funciones, clases e instancias de clases.
- **Herramientas de IA:**

Objetivo de uso	Aprender a manejar datos del tipo JSON en Python
Herramienta utilizada	Google Gemini
Prompt o pregunta	Chat, genera una clase que logre leer y guardar datos en un archivo JSON. El cuál tiene que seguir la siguiente estructura: { "gamerX": [{ "dificultad": "difícil", "tiempo": 5600, "fecha_hora": "20260508T13:15:40"}, { "dificultad": "fácil", "tiempo": 3000, "fecha_hora": "20260508T16:00:30"}, { "dificultad": "difícil", "tiempo": 4000, "fecha_hora": "20260508T21:10:30"}, ...] "gamer10": [{ }, { }, ...] }
Respuesta	Ver anexo 1
¿Cómo usó o adaptó la respuesta?	Se extrajo el archivo creado y se implementó en el programa, se revisó que partes funcionaban, eliminando detalles innecesarios como prints, o la manera en la que extraía los datos, quedando al final una función save para guardar los datos, read para leer el archivo e inicialmente registrar_partida.
Reflexión crítica	La IA logró implementar rápidamente el apartado de lectura y escritura de archivos además de explicar un poco su funcionamiento. Esto ahorró mucho tiempo en investigación, implementación y corrección de errores de formato.
Objetivo de uso	Crear un documento visualmente atractivo a la vista que refleje los datos almacenados en el JSON de partidas.

Herramienta utilizada	Google Gemini
Prompt o pregunta	Por medio de esta librería fpdf cree un archivo que: • Amplía cada texto para que quede con en columnas de 3 • Haz las mismas líneas para NIVEL: FACIL e NIVEL:INTERMEDIO • Entre cada linea despiega los datos del json sudoku2026_bitácora_jugadas según el orden anterior, si no hay jugadas de ese nivel simplemente deja ese lado vacío
Respuesta	Anexo 2
¿Cómo usó o adaptó la respuesta?	La IA generó una función general donde solo adapté el apartado de modificación del documento PDF, obviando la extracción de datos que podía implementar usando las funciones del JSONParser previamente hechas.
Reflexión crítica	Al igual que con los archivos JSON, esto ahorró mucho tiempo de debug y solución de errores, la librería si tenía conocimiento de uso, sin embargo para lograr un resultado visual bonito se recurrió a esta herramienta.

Objetivo de uso	Comprender el uso del módulo Pickle con un ejemplo sencillo
Herramienta utilizada	Google Gemini
Prompt o pregunta	Realiza una prueba, crea una clase en python que escriba los siguientes datos Nombre, Apellido, telefono y cree un archivo .pickle y lo lea
Respuesta	Anexo 3
¿Cómo usó o adaptó la respuesta?	La IA creó una clase que usé como plantilla, la cuál se encargaba de leer el archivo pickle almacenado y guardarlo con nuevos datos. Con el sistema ya montado logré modificarlo a mis necesidades para guardar el árbol de jugadas.
Reflexión crítica	Para entender como se usa una clase o módulo de manera sencilla es muy útil, similar a ver un tutorial del mismo pero sin la necesidad de copiar el código a mano y más directo.

Objetivo de uso	Comprender el funcionamiento de la clase que envía mails.
Herramienta utilizada	Google Gemini
Prompt o pregunta	Crea una clase que al iniciarla envíe un correo diciendo "hello world" a esta dirección de correo ezeproc2005@gmail.com
Respuesta	Anexo 4
¿Cómo usó o adaptó la respuesta?	Gracias a la clase construida pude comprender la estructura utilizada y logré modificar los mensajes además de modificar las funciones para que se lograra enviar desde mi cuenta de correo el código de ingreso a los correos que el usuario ingrese.
Reflexión crítica	Al igual que la clase anterior resulta extremadamente útil para aprender a usar clases por primera vez, en este caso como utiliza internet, ahorra todo el apartado de los protocolos y conexiones, logrando que nos centremos en su funcionalidad en el programa

Estructura archivos usados:

El programa contiene una carpeta central llamada “Programa_II” la cuál contiene todos los archivos necesarios del programa, tanto archivos de ejecución (Python) así como de configuración (JSON). Adicionalmente, debido a la ejecución, crea dos carpetas llamadas “vscode” y “_pycache_” las cuales se encargan de mantener una configuración de arranque predeterminada y archivos temporales para agilizar la ejecución respectivamente.

Conclusiones del trabajo:

Problemas encontrados y soluciones implementadas.

La mayoría de problemas encontrados realmente se abordaron como retos o implementaciones necesarias para lograr cumplir con los requerimientos de programa. Entre ellos tenemos la representación del tablero, la lectura y escritura de datos, la generación de archivos PDF y las posibilidades de configuración en cara al usuario.

Para lo primero, la complejidad implicaba encontrar un sistema que lograra almacenar datos de manera tabular, además de cumplir reglas propias del sudoku como:

- No repetir números en la fila y columna.
- Cada subgrupo de 3x3 casillas no puede tener ningún número repetido.

Para lograr esto, se utilizó una matriz, especial para representaciones numéricas de tablas. Además de comprobaciones con ciclos para comprobar repeticiones en la fila y columna del elemento seleccionado más un ciclo que analiza el subgrupo partiendo desde el origen del mismo según el elemento y recorriendo 3 columnas y 3 filas.

Para la representación de datos se debía partir de un formato fácil de usar, leer, interpretar y escribir. La solución fue usar el formato JSON el cuál posee

mucha documentación en internet, es fácil de implementar, posee librerías de Python para lograr leerlos y está estandarizado.

Finalmente se decidió re utilizar la librería fpdf de proyectos pasados, debido a una fácil curva de aprendizaje al ya haberse trabajado con esta tecnología con anterioridad. Esta librería permite escribir los datos en filas que se escriben sobre el PDF.

Para este segundo avance se encontraron menos dificultades a la hora de implementar sus funcionalidades, la más destacable, la confusión de poder almacenar clases en un archivo en vez de datos, con el módulo pickle, y de leerlos y convertirlos a objetos dentro del progra

Estadística tiempos:

Actividad realizada	Horas
Análisis problema	50 min
Diseño de algoritmos	2 horas
Marco teórico	40 min
Programación	7 horas y 30 min
Documentación interna	1 hora y 22 minutos
Pruebas	1 hora
Elaboración manual de usuario	2 horas
Total	15 horas y 22 min

Actividad realizada	Horas
Análisis problema	2 horas
Diseño de algoritmos	3 horas
Marco teórico	1 hora
Programación	3 horas
Documentación interna	2 horas
Pruebas	30 min
Elaboración manual de usuario	1 hora
Total	14 horas y 30 min

Lista revisión proyecto:

Concepto	Puntos Originales	Avance	Puntos obtenidos	Análisis
Menú	1	100%		
Opción jugar	15	100%		
Botón iniciar juego	10	100%		
Crear top x	5	100%		
Botón deshacer jugada	8	90%		No se implementó la pila con librería externa sino con las listas genéricas de Python.
Botón rehacer jugada	8	90%		No se implementó la pila con librería externa sino con las listas genéricas de Python.
Botón borrar juego	3	100%		
Botón terminar juego	3	100%		
Botón Top x	6	100%		
Formato PDF	2	100%		
Botón Guardar Juego	8	100%		
Botón Cargar Juego incluyendo su despliegue	8	100%		
Opción configurar	8	100%		
Ayuda (Manual de usuario)	5	100%		
Acerca de	1	100%		
Salir	1	100%		
Relog: Cronómetro y timer tiempo real	8	100%		

Concepto	Puntos Originales	Avance	Puntos obtenidos	Análisis
Esquema de cuentas para acceder a la aplicación	20	100%		
Generar partidas automáticamente	10	100%		
Panel de elementos	10	90%		Se podría mejorar la manera en la que se indica al usuario.
Juegos multinivel	10	90%		Se podría mejorar su funcionamiento, el guardar partidas y cargarlas.
Cambios para el manejo del Top X	40	100%		
Funcionalidad definida por el programador	10	100%		

Anexos:

- Anexo 1:

¡Me parece excelente! Cambiar el foco a la gestión del historial es una gran idea, ya que te permitirá guardar las puntuaciones y los tiempos del cronómetro que acabamos de armar.

Para manejar esto de forma limpia y ordenada (siguiendo las buenas prácticas de ingeniería de software), lo ideal es crear una clase independiente encargada exclusivamente de la persistencia de datos. Así, tu interfaz gráfica no se mezcla con la lógica de guardar archivos.

Aquí tienes la implementación de la clase `GestorHistorialJSON` diseñada específicamente para la estructura que necesitas:

Clase `GestorHistorialJSON`

```

1 import json
2 import os
3 from datetime import datetime
4
5 class GestorHistorialJSON:
6     def __init__(self, nombre_archivo="historial_sudoku.json"):
7         self.nombre_archivo = nombre_archivo
8         # Nos aseguramos de que el archivo exista al instanciar la clase
9         self._verificar_archivo()
10
11     def _verificar_archivo(self):
12         """Método interno para crear el archivo con la estructura inicial si no existe."""
13         if not os.path.exists(self.nombre_archivo):
14             # Estructura base que me pediste: un diccionario con la clave "jugador" y una lista vacía
15             estructura_inicial = {"jugador": []}
16             self._guardar_todo(estructura_inicial)
17
18     def _leer_todo(self):
19         """Lee el archivo JSON completo y lo devuelve como un diccionario de Python."""
20         try:
21             with open(self.nombre_archivo, "r", encoding="utf-8") as archivo:
22                 return json.load(archivo)
23         except (json.JSONDecodeError, FileNotFoundError):
24             # Si el archivo está corrupto o vacío, devolvemos la estructura limpia
25             return {"jugador": []}
26
27     def _guardar_todo(self, datos):
28         """Escribe un diccionario de Python en el archivo JSON."""
29         with open(self.nombre_archivo, "w", encoding="utf-8") as archivo:
30             # indent=4 hace que el archivo JSON sea legible para humanos (con saltos de línea)
31             json.dump(datos, archivo, ensure_ascii=False, indent=4)
32
33     def registrar_partida(self, dificultad, tiempo):
34         """
35         Agrega una nueva partida al historial del jugador.
36         La fecha y hora se calculan automáticamente en el momento del registro.
37         """
38         # 1. Capturamos la fecha y hora actual con un formato limpio (AAAA-MM-DD HH:MM:SS)
39         fecha_hora_actual = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
40
41         # 2. Creamos el nuevo registro
42         nueva_jugada = {
43             "dificultad": dificultad,
44             "tiempo": tiempo, # Aquí pasarás el string del cronómetro "HH:MM:SS"
45             "fecha_hora": fecha_hora_actual
46         }
47
48         # 3. Leemos lo que ya estaba guardado
49         datos = self._leer_todo()
50
51         # 4. Añadimos la nueva jugada a la lista de la clave "jugador"
52         datos["jugador"].append(nueva_jugada)
53
54         # 5. Volvemos a guardar el archivo actualizado
55         self._guardar_todo(datos)
56         print("¡Partida registrada con éxito en el JSON!")
57
58     def obtener_historial(self):
59         """Devuelve la lista completa de partidas del jugador."""
60         datos = self._leer_todo()
61         return datos.get("jugador", [])
62

```

¿Cómo puedes usar esta clase dentro de tu Sudoku?

Es sumamente sencillo. Te muestro cómo integrarla en tu flujo actual:

1. Instanciarla al inicio

En el `__init__` de tu ventana de Tkinter, creas un objeto de esta clase:

```

1 def __init__(self):
2     super().__init__()
3     # ... tus otras variables (self.segundos_totales, etc.) ...
4
5     # Creamos el gestor de archivos
6     self.gestor_json = GestorHistorialJSON()
7
8     self.createIGU()
9

```

2. Guardar la partida cuando el usuario gane

Cuando detectes que el Sudoku se completó correctamente, pausas el cronómetro y mandas a guardar los datos usando el texto del reloj:

```

1 def juego_ganado(self):
2     self.pausar_crono() # Detiene el reloj
3
4     # Supongamos que recuperas el tiempo formateado de tu label
5     tiempo_final = self.lbl_reloj.cget("text")
6     dificultad_actual = "Fácil" # O la dificultad que tengas activa
7
8     # Guardamos en el JSON de forma automática
9     self.gestor_json.registrar_partida(dificultad_actual, tiempo_final)
10

```

¿Cómo se verá el archivo generado (historial_sudoku.json)?

Se creará automáticamente en la misma carpeta de tu proyecto y lucirá exactamente con la estructura que solicitaste:

```

1 {
2     "jugador": [
3         {
4             "dificultad": "Fácil",
5             "tiempo": "00:14:25",
6             "fecha_hora": "2026-06-04 02:08:00"
7         },
8         {
9             "dificultad": "Medio",
10            "tiempo": "00:08:12",
11            "fecha_hora": "2026-06-04 02:25:14"
12        }
13    ]
14 }
15

```

- Anexo 2:

Para lograr este reporte estructurado en columnas y clasificado por niveles utilizando fpdf, primero debemos leer tu archivo JSON de bitácora, filtrar las jugadas por su respectivo nivel ("Facil", "Medio" o "Intermedio", y "Dificil") y luego usar el método cell() con anchos fijos para simular las columnas de una tabla.

Dado que la librería clásica de FPDF tiene problemas con caracteres con acentos (como la Í de DIFÍCIL, la Á de BITÁCORA o la Ó), aplicaremos un formateo seguro convirtiendo los textos a latin-1 para que no te genere errores ni caracteres extraños en el PDF.

Aquí tienes el código completo y automatizado para leer tu JSON y generar el reporte:

```
1 import json
2 from fpdf import FPDF
3 from tkinter import messagebox
4
5 def generar_reporte_pdf():
6     # 1. Intentar leer el archivo JSON de la bitácora
7     try:
8         with open("sudoku2026_bitacora_jugadas.json", "r", encoding="utf-8") as archivo:
9             jugadas = json.load(archivo)
10        except FileNotFoundError:
11            messagebox.showerror("Error", "No se encontró el archivo 'sudoku2026_bitacora_jugadas.json'")
12            return
13        except json.JSONDecodeError:
14            # Si el archivo está vacío o mal estructurado, inicializamos una lista vacía
15            jugadas = []
16
17    # 2. Inicializar el objeto PDF (Formato Carta, unidades en milímetros)
18    pdf = FPDF(orientation="P", unit="mm", format="letter")
19    pdf.add_page()
20
21    # Configurar título principal del PDF
22    pdf.set_font("Arial", style="B", size=15)
23    titulo = "BITÁCORA DE JUGADAS - SUDOKU 2026"
24    pdf.cell(w=0, h=12, txt=titulo.encode('latin-1', 'ignore').decode('latin-1'), ln=1, align='C')
25    pdf.ln(10) # Espacio en blanco
26
27    # 3. Definir los niveles que vamos a evaluar de manera secuencial
28    # Nota: Ajusta "Medio" o "Intermedio" según cómo lo guardes textualmente en tu JSON
29    niveles = ["Facil", "Intermedio", "Dificil"]
30
31    # Definir el ancho de las 3 columnas (El ancho total de una página carta es ~215mm)
32    # Dejando márgenes, podemos asignar sima a cada columna de datos
33    ancho_col1 = 60 # Columna JUGADOR
34    ancho_col2 = 60 # Columna TIEMPO JUGADO
35    ancho_col3 = 60 # Columna EL (FECHA)
36
37    for nivel in niveles:
38        # Configurar fuente para el encabezado del nivel (Negrita, tamaño 12)
39        pdf.set_font("Arial", style="B", size=12)
40
41        # Formatear el nombre del nivel visualmente
42        nivel_encabezado = f"NIVEL: {nivel.upper()}"
43        pdf.cell(w=0, h=0, txt=nivel_encabezado.encode('latin-1', 'ignore').decode('latin-1'), ln=1, align='L')
44
45        # Crear los títulos de las 3 columnas debajo de cada nivel
46        pdf.set_font("Arial", style="B", size=10)
47        pdf.cell(w=ancho_col1, h=0, txt="JUGADOR".encode('latin-1', 'ignore').decode('latin-1'), ln=0, align='L')
48        pdf.cell(w=ancho_col2, h=0, txt="TIEMPO JUGADO".encode('latin-1', 'ignore').decode('latin-1'), ln=0, align='L')
49        pdf.cell(w=ancho_col3, h=0, txt="EL".encode('latin-1', 'ignore').decode('latin-1'), ln=0, align='L')
50
51        # Dibujar una línea sutil de división debajo de los títulos de columna
52        pdf.ln(pdf.get_x(), pdf.get_y(), pdf.get_x() + 180, pdf.get_y())
53        pdf.ln(2)
54
55        # 4. Filtrar y desplegar las jugadas que pertenecen a este nivel específico
56        pdf.set_font("Arial", style="N", size=10) # Fuente normal para los datos
57        jugadas_encontradas = False
58
59        for jugada in jugadas:
60            # Comparamos ignorando mayúsculas/minúsculas por seguridad
61            if jugada.get("dificultad", "").lower() == nivel.lower():
62                jugadas_encontradas = True
63
64            # Extraemos los datos del JSON (asegurado de que estas llaves coincidan con tu JSON)
65            nombre_jugador = str(jugada.get("jugador", "Desconocido"))
66            tiempo_jugado = str(jugada.get("tiempo_jugado", "00:00:00"))
67            fecha_jugada = str(jugada.get("fecha", "--/--/----"))
68
69            # Imprimimos las 3 celdas alineadas en la misma fila (ln=0 para las primeras, ln=1 para la última)
70            pdf.cell(w=ancho_col1, h=0, txt=nombre_jugador.encode('latin-1', 'ignore').decode('latin-1'), ln=0, align='L')
71            pdf.cell(w=ancho_col2, h=0, txt=tiempo_jugado.encode('latin-1', 'ignore').decode('latin-1'), ln=0, align='L')
72            pdf.cell(w=ancho_col3, h=0, txt=fecha_jugada.encode('latin-1', 'ignore').decode('latin-1'), ln=0, align='L')
73
74        # Si no hay registros para este nivel, dejamos el espacio vacío con una pequeña notificación discreta
75        if not jugadas_encontradas:
76            pdf.set_font("Arial", style="I", size=10) # letra cursiva
77            pdf.cell(w=0, h=0, txt="No hay jugadas registradas en este nivel.".encode('latin-1', 'ignore').decode('latin-1'), ln=1, align='L')
78
79        # Añadir un espacio amplio antes de pasar al siguiente bloque de nivel
80        pdf.ln(10)
81
82    # 5. Guardar el archivo PDF resultante
83    try:
84        pdf.output("reporte_bitacora_sudoku.pdf")
85        messagebox.showinfo("Reporte", "¡El archivo PDF de la bitácora fue generado con éxito!")
86    except Exception as e:
87        messagebox.showerror("Error", f"No se pudo guardar el PDF. Asegúrate de que no esté abierto.\nError: {e}")
88
89    # Llamar a la función para probar la generación del archivo
90    # generar_reporte_pdf()
```

Puntos clave de esta estructura:

1. **Manejo de columnas con `ln=0` y `ln=1`:** Para armar las filas de 3 columnas, las dos primeras celdas llevan `ln=0` para que el cursor de `fpdf` se quede al lado derecho de la celda y sigan en la misma línea. La última columna lleva `ln=1` para hacer el salto de línea definitivo hacia abajo.
2. **Control de datos vacíos:** El script utiliza la bandera `bohemia` jugadas_encontradas. Si recorre todo tu JSON y no halla partidas que coincidan con "Facil", pondrá el mensaje elegante en cursiva y avanzará al siguiente nivel, manteniendo el reporte limpio.
3. **Escudo `.encode('latin-1', 'ignore')`:** Esto evita que el programa se caiga si un jugador registró su nombre con tildes o caracteres especiales, asegurando que el PDF siempre se escriba correctamente.

- Anexo 3:

¡Hagamos la prueba! Aquí tienes el código de una clase en Python llamada `GestorContactos`. Esta clase utiliza la librería nativa `pickle` para serializar (escribir) un diccionario con los datos del usuario en un archivo binario y luego deserializarlo (leerlo) de vuelta a la memoria.

```
1 import pickle
2 import os
3
4 class GestorContactos:
5     def __init__(self, nombre_archivo="contacto.pkl"):
6         self.nombre_archivo = nombre_archivo
7
8     def guardar_contacto(self, nombre, apellido, telefono):
9         """Recibe los datos, los empaqueta en un diccionario y los escribe usando Pickle."""
10        datos = {
11            "Nombre": nombre,
12            "Apellido": apellido,
13            "Telefono": telefono
14        }
15
16        # 'wb' significa Write Binary (Escritura Binaria), necesaria para pickle
17        with open(self.nombre_archivo, "wb") as archivo:
18            pickle.dump(datos, archivo)
19        print(f"✅ Datos guardados exitosamente en '{self.nombre_archivo}'.")
20
21    def leer_contacto(self):
22        """Lee el archivo .pickle y reconstruye el objeto original en memoria."""
23        if not os.path.exists(self.nombre_archivo):
24            print(f"❌ El archivo '{self.nombre_archivo}' no existe todavía.")
25            return None
26
27        # 'rb' significa Read Binary (Lectura Binaria)
28        with open(self.nombre_archivo, "rb") as archivo:
29            datos_cargados = pickle.load(archivo)
30        return datos_cargados
31
32 # =====
33 # 🟢 PRUEBA DE EJECUCIÓN
34 # =====
35 if __name__ == "__main__":
36     # 1. Instanciar la clase
37     gestor = GestorContactos()
38
39     print("--- 1. Guardando datos en el archivo .pickle ---")
40     gestor.guardar_contacto(
41         nombre="Ezequiel",
42         apellido="Bonilla",
43         telefono="8888-5555"
44     )
45
46     print("\n--- 2. Leyendo datos desde el archivo .pickle ---")
47     resultado = gestor.leer_contacto()
48
49     if resultado:
50         print("Datos recuperados con éxito:")
51         print(f"• Nombre: {resultado['Nombre']}")
52         print(f"• Apellido: {resultado['Apellido']}")
53         print(f"• Teléfono: {resultado['Telefono']}")
54
55     # Prueba de tipo de dato
56     print(f"📄 Tipo de dato recuperado: {type(resultado)}")
```

Ventajas que notarás para tu Sudoku:

1. **Preservación de Tipos:** Si abres el archivo contacto.pkl con un bloc de notas, verás caracteres extraños (código binario). Esto pasa porque pickle no escribe texto plano como JSON; guarda la estructura exacta de la memoria.
 2. **Cero conversión de datos:** Al leer el archivo con pickle.load(), te devuelve un objeto con el tipo de dato exacto de Python (dict, list, etc.). Las llaves numéricas de tus matrices o mapeos seguirán siendo números enteros (int) sin transformarse a strings automáticamente.
- Anexo 4:

Para enviar correos electrónicos desde Python, la forma estándar y nativa es utilizando la librería smtplib junto con el módulo email.mime.

Dado que estás construyendo una arquitectura orientada a objetos para tu Sudoku, aquí tienes una clase limpia llamada EnvioCorreo. Al instanciarla (cuando se ejecuta su constructor __init__), intentará conectarse al servidor SMTP y enviar el mensaje automáticamente.

```
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

class EnvioCorreo:
    def __init__(self):
        # 1. Configuración de credenciales y servidor (Ejemplo con Gmail)
        self.remite = "tu_correo_aplicacion@gmail.com"
        # ¡OJO! Esta contraseña debe ser una "Contraseña de aplicación" generada en tu cuenta de Google
        self.password = "tu_contraseña_de_aplicacion_aqui"

        self.destinatario = "ezebroch2005@gmail.com"
        self.smtp_server = "smtp.gmail.com"
        self.port = 587 # Puerto estándar para TLS

        # 2. Ejecutar el envío de forma inmediata al iniciar la clase
        self.enviar_hello_world()

    def enviar_hello_world(self):
        try:
            # 3. Crear la estructura del mensaje
            msg = MIMEMultipart()
            msg['From'] = self.remite
            msg['To'] = self.destinatario
            msg['Subject'] = "Código de Acceso - Sudoku 2026"

            # Cuerpo del mensaje
            cuerpo_mensaje = "hello world"
            msg.attach(MIMEText(cuerpo_mensaje, 'plain'))

            # 4. Conectarse de forma segura al servidor SMTP
            print(f"■ Conectando al servidor de correo para {self.destinatario}...")
            server = smtplib.SMTP(self.smtp_server, self.port)
            server.starttls() # Cifrado seguro

            # Iniciar sesión y enviar
            server.login(self.remite, self.password)
            server.sendmail(self.remite, self.destinatario, msg.as_string())

            # Cerrar conexión
            server.quit()
            print("■ Correo enviado exitosamente de manera automatizada.")

        except Exception as e:
            print(f"✗ Error crítico al intentar enviar el correo: {e}")

# --- Cómo ejecutarla en tu programa ---
if __name__ == "__main__":
    # Solo con instanciar la clase, el constructor __init__ se encarga de todo el flujo
    correo_sistema = EnvioCorreo()
```

Configuración obligatoria para que funcione (Evitar bloqueos de Google)

Si intentas usar la contraseña normal con la que inicias sesión en tu Gmail, Google va a bloquear la conexión por motivos de seguridad (SMTPAuthenticationError). Para solucionarlo en tu entorno de desarrollo, sigue estos pasos:

1. Ve a la configuración de tu cuenta de Google (la cuenta que usará el juego para despachar los mensajes).
2. Asegúrate de tener activada la **Verificación en dos pasos**.
3. Busca en la barra de búsqueda superior interna de la configuración el término: **"Contraseñas de aplicaciones"** (App Passwords).
4. Dale un nombre (ej: "Sistema Sudoku Python") y genera una clave.
5. Google te dará un código de **16 caracteres**. Copia ese código exacto y pégalo en la variable `self.password` de tu script.

Consejo para tu feature de Login:

Cuando integres esto con tu generador de códigos aleatorios de 6 dígitos, solo tendrás que modificar esta clase para que reciba el código numérico por parámetro y lo concatene en la variable

```
1 # Ejemplo futuro:
2 # class EnvioCorreo:
3 #     def __init__(self, codigo_otp):
4 #         ...
5 #         cuerpo_mensaje = f"Tu código de ingreso único al Sudoku es: {codigo_otp}"
```

Referencias:

- Estandarte. (2026) Archivos PDF: qué son y para qué se utilizan. Estandarte.
https://www.estandarte.com/noticias/varios/archivos-pdf-que-son-y-para-que-se-utilizan_4797.html
- Moore, A. D. (2018). Python GUI Programming with Tkinter. Packt Publishing.
- IBM. (2025). Origen de datos. IBM Documentation.
<https://www.ibm.com/docs/es/i/7.5.0?topic=files-data-source>
- Dabek, F., Zeldovich, N., Kaashoek, F., Mazières, D., & Morris, R. (2002). Event-driven Programming for Robust Software. En Proceedings of the ACM SIGOPS European Workshop (pp. 186-189). ACM Laboratory for Computer Science.
<https://doi.org/10.1145/1133373.1133410>
- Luna González, L. J. (2004). El diseño de interfaz gráfica de usuario para publicaciones digitales. Revista Digital Universitaria, 5(7).
https://revista.unam.mx/vol.5/num7/art44/ago_art44.pdf
- Pompa Sourd, F. (2022). XML, JSON y el intercambio de información. *Revista Ciencia Universitaria*, 18(3). Recuperado a partir de
<https://revistas.unah.edu.cu/index.php/cu/article/view/1532>
- Google. (2026). Gemini (Versión de junio de 2026) [Modelo de lenguaje grande].
<https://gemini.google.com/>
- W3Schools. (s.f.). *Data Structures and Algorithms - Binary Tree*. Recuperado el 28 de junio de 2026, de
https://www.w3schools.com/dsa/dsa_data_binarytrees.php
- Pilgrim, M. (2009). Serializing Python Objects. In: Dive Into Python 3. Apress.
https://doi.org/10.1007/978-1-4302-2416-7_13
-